

Capítulo 8

Chapter Precap

- Comprobaciones predictivas previas y la importancia de estas para la construcción de modelos.
- Cómo especificar probabilidades priores más específicas para parámetros individuales de un modelo.
- Introducimos modelos heterocedásticos, es decir, modelos con términos de error que varían de una observación a otra.
- Presentamos un modelo "simple" que incluye solo variación en el término de error para diferentes condiciones.
- Presentamos un modelo "complejo" que presenta términos de error dependientes del oyente, además del equivalente de 'efectos aleatorios' para el término de error.
- Finalmente, discutimos la construcción de modelos identificables, modelos respaldados por los datos disponibles, colinealidad, dependencia lineal y modelos saturados.

More About Priors

- En muchos casos (en lingüística) los priores tendrán poco o ningún efecto práctico.
- Utiliza conocimiento del dominio para obtener priores que estén mas o menos cerca: ¿12 años luz o 12 nanómetros?
- A veces esto es fácil (la altura humana), a veces no es tan fácil (el efecto de la frecuencia léxica en los tiempos de reacción logarítmicos).

Prior Predictive Checks

- Crea un muestreo de datos falso solo a partir del prior (ignore la verosimilitud, es decir, los datos).

```
# Fit the model yourself
priors = c(brms::set_prior("student_t(3,156, 1000)", class = "Intercept"),
           brms::set_prior("student_t(3,0, 1000)", class = "b"),
           brms::set_prior("student_t(3,0, 1000)", class = "sd"),
           brms::set_prior("lkj_corr_cholesky (1000)", class = "cor"),
           brms::set_prior("student_t(3,0, 1000)", class = "sigma"))

prior_uninformative =
  brms::brm (height ~ A + G + A:G + (A + G + A:G|L) + (1|S),
             sample_prior="only", data = exp_data, chains = 4, cores = 4,
            warmup = 1000, iter = 5000, thin = 4, prior = priors)
```

Prior Predictive Checks

Uninformative
Priors



```
priors = c(brms::set_prior("student_t(3,156, 1000)", class = "Intercept"),  
          brms::set_prior("student_t(3,0, 1000)", class = "b"),  
          brms::set_prior("student_t(3,0, 1000)", class = "sd"),  
          brms::set_prior("lkj_corr_cholesky (1000)", class = "cor"),  
          brms::set_prior("student_t(3,0, 1000)", class = "sigma"))
```

Mildly
informative
Priors



```
priors = c(brms::set_prior("student_t(3,156, 12)", class = "Intercept"),  
          brms::set_prior("student_t(3,0, 12)", class = "b"),  
          brms::set_prior("student_t(3,0, 12)", class = "sd"),  
          brms::set_prior("lkj_corr_cholesky (12)", class = "cor"),  
          brms::set_prior("student_t(3,0, 12)", class = "sigma"))
```

Conservative
Priors



```
priors = c(brms::set_prior("student_t(3,156, 6)", class = "Intercept"),  
          brms::set_prior("student_t(3,0, 6)", class = "b"),  
          brms::set_prior("student_t(3,0, 6)", class = "sd"),  
          brms::set_prior("lkj_corr_cholesky (2)", class = "cor"),  
          brms::set_prior("student_t(3,0, 6)", class = "sigma"))
```

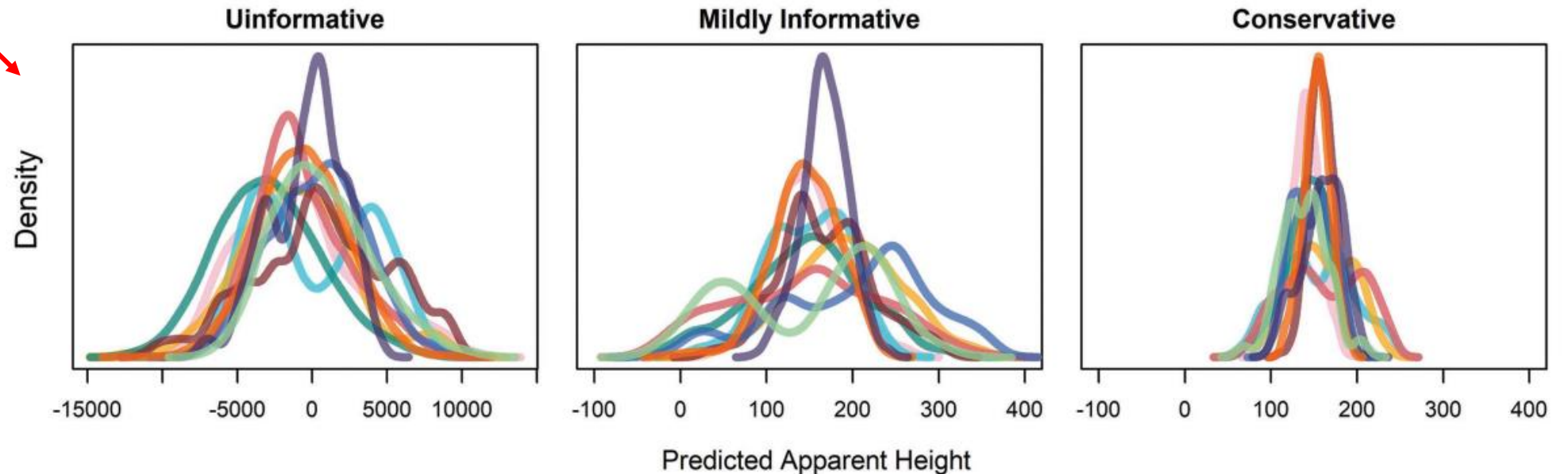
Prior Predictive Checks

```
pp_uninformative = predict (prior_uninformative, summary = FALSE)
```

```
hist (pp_uninformative[1,])
```

```
bmmb::p_check (pp_uninformative)
```

No se usan los
datos. Solo en base
al conocimiento del
dominio.

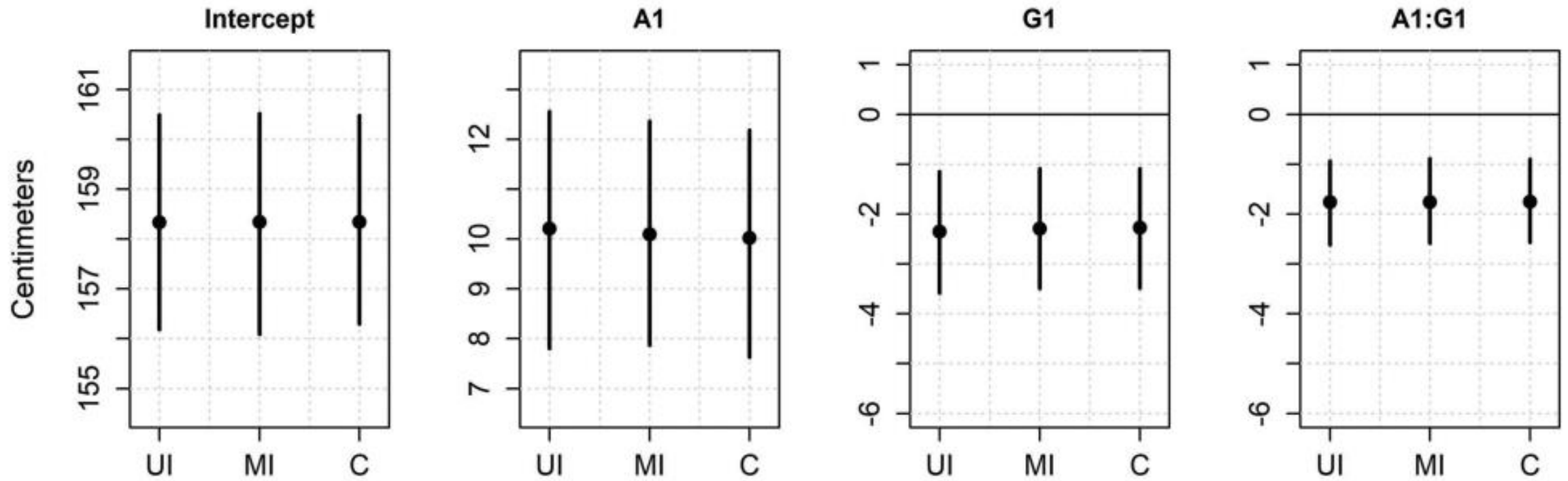


No Effect on Results!

UI: Uninformative

MI: Mildly informative

C: Conservative



More Specific Priors

```
# we omit empty columns to let the output fit on the page
bmmmb::prior_summary(model_mildly_informative)[,-c(5:9)]
```

##	prior	class	coef	group	source
##	student_t(3,0, 12)	b			user
##	student_t(3,0, 12)	b	A1		default
##	student_t(3,0, 12)	b	A1:G1		default
##	student_t(3,0, 12)	b	G1		default
##	student_t(3,156, 12)	Intercept			user
##	lkj_corr_cholesky (2)	L			user
##	lkj_corr_cholesky (2)	L		L	default
##	student_t(3,0, 12)	sd			user
##	student_t(3,0, 12)	sd		L	default
##	student_t(3,0, 12)	sd	A1	L	default
##	student_t(3,0, 12)	sd	A1:G1	L	default
##	student_t(3,0, 12)	sd	G1	L	default
##	student_t(3,0, 12)	sd	Intercept	L	default
##	student_t(3,0, 12)	sd		S	default
##	student_t(3,0, 12)	sd	Intercept	S	default
##	student_t(3,0, 12)	sigma			user

More Specific Priors

```
priors =
  c(set_prior("student_t(3,156, 6)", class = "Intercept"),
    set_prior("student_t(3,0, 6)", class = "b"),
    set_prior("student_t(3,0, 10)", class = "b", coef = "A1"),
    set_prior("student_t(3,0, 3)", class = "b", coef = "G1"),
    set_prior("student_t(3,0, 10)", class = "sd"),
    set_prior("student_t(3,0, 5)", class = "sd", coef = "A1", group="L"),
    set_prior("student_t(3,0, 1.5)", class = "sd", coef = "G1", group="L"),
    set_prior("lkj_corr_cholesky (2)", class = "cor"),
    set_prior("student_t(3,0, 6)", class = "sigma"))
```

```
bmmb::prior_summary(prior_informative)[,-c(5:8)]
```

##	prior	class	coef	group
##	student_t(3,0, 6)	b		
##	student_t(3,0, 10)	b	A1	
##	student_t(3,0, 10)	b	A1:G1	
##	student_t(3,0, 3)	b	G1	
##	student_t(3,156, 6)	Intercept		
##	lkj_corr_cholesky (2)	L		
##	lkj_corr_cholesky (2)	L		L
##	student_t(3,0, 10)	sd		
##	student_t(3,0, 10)	sd		L
##	student_t(3,0, 5)	sd	A1	L
##	student_t(3,0, 5)	sd	A1:G1	L
##	student_t(3,0, 1.5)	sd	G1	L
##	student_t(3,0, 1.5)	sd	Intercept	L
##	student_t(3,0, 1.5)	sd		S
##	student_t(3,0, 1.5)	sd	Intercept	S
##	student_t(3,0, 6)	sigma		

Data and Research Questions

```
library (brms)
library (bmmb)
data (exp_data)
options (contrasts = c('contr.sum', 'contr.sum'))
```

- **L**: A number from 1 to 15 indicating which *listener* responded to the trial.
- **height**: A number representing the *height* (in centimeters) reported for the speaker on each trial.
- **S**: A number from 1 to 139 indicating which *speaker* produced the trial stimulus.
- **G**: The *apparent gender* of the speaker indicated by the listener, **f** (female) or **m** (male).
- **A**: The *apparent age* of the speaker indicated by the listener, **a** (adult) or **c** (child).

(Q1) Does our error standard deviation vary as a function of apparent speaker age?

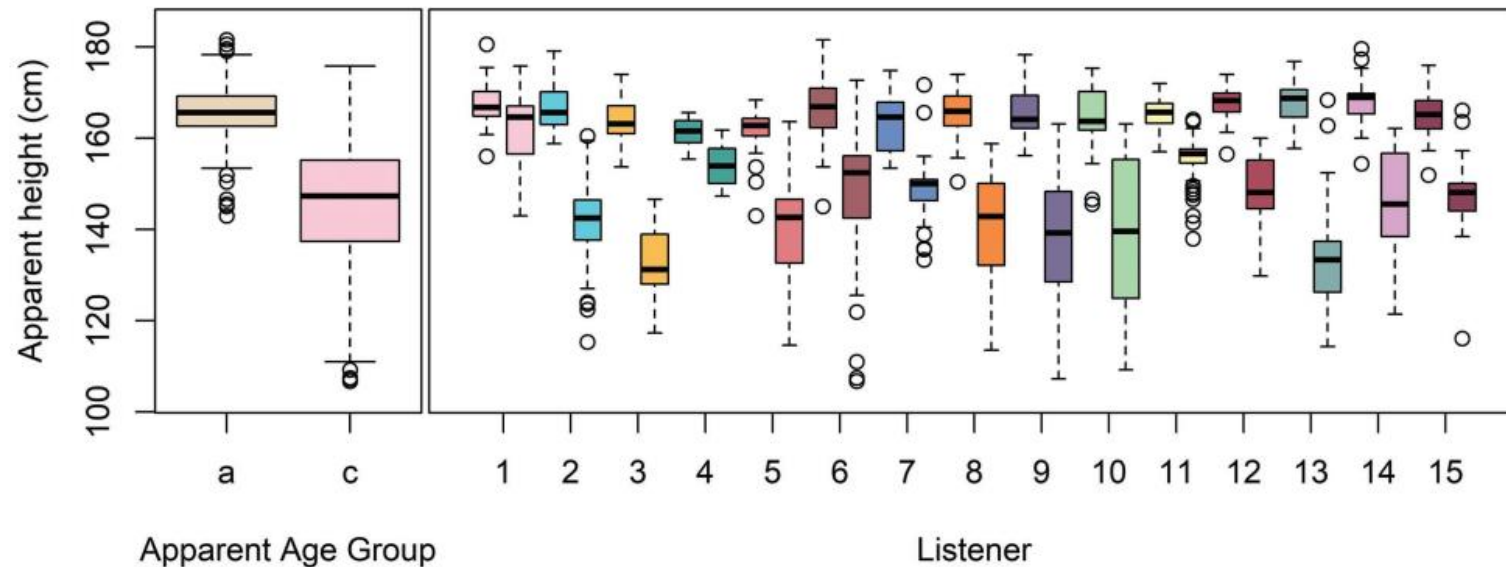
(Q2) Does our error standard deviation vary as a function of the listener?

Homoskedasticity

- La mayoría de los modelos de regresión solo predicen la variación de las medias.
- Se supone una única varianza de error para todos los datos (es decir, σ no obtiene un subíndice).

$$\text{height}_{[i]} \sim \mathcal{N}(\mu_{[i]}, \sigma)$$

$$\mu_{[i]} = x_1 + x_2 + \dots + x_p$$



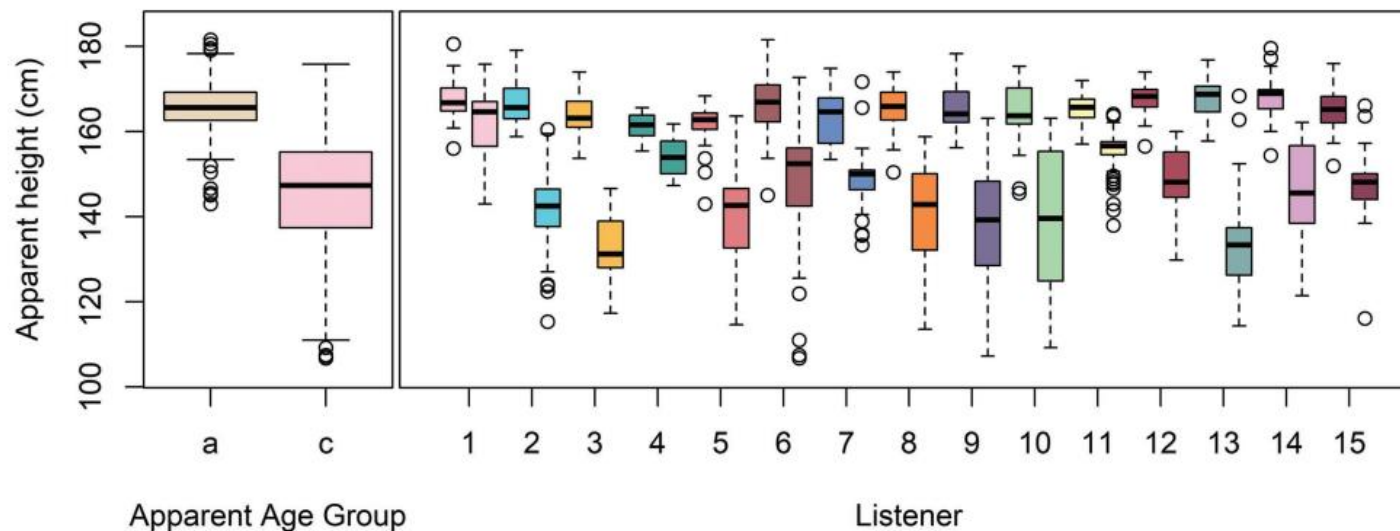
Heteroskedasticity

- Con los modelos bayesianos, es trivial modelar variación en las varianzas de error.
- Pueden existir diferentes variaciones de error para diferentes datos (es decir, σ obtiene un subíndice).

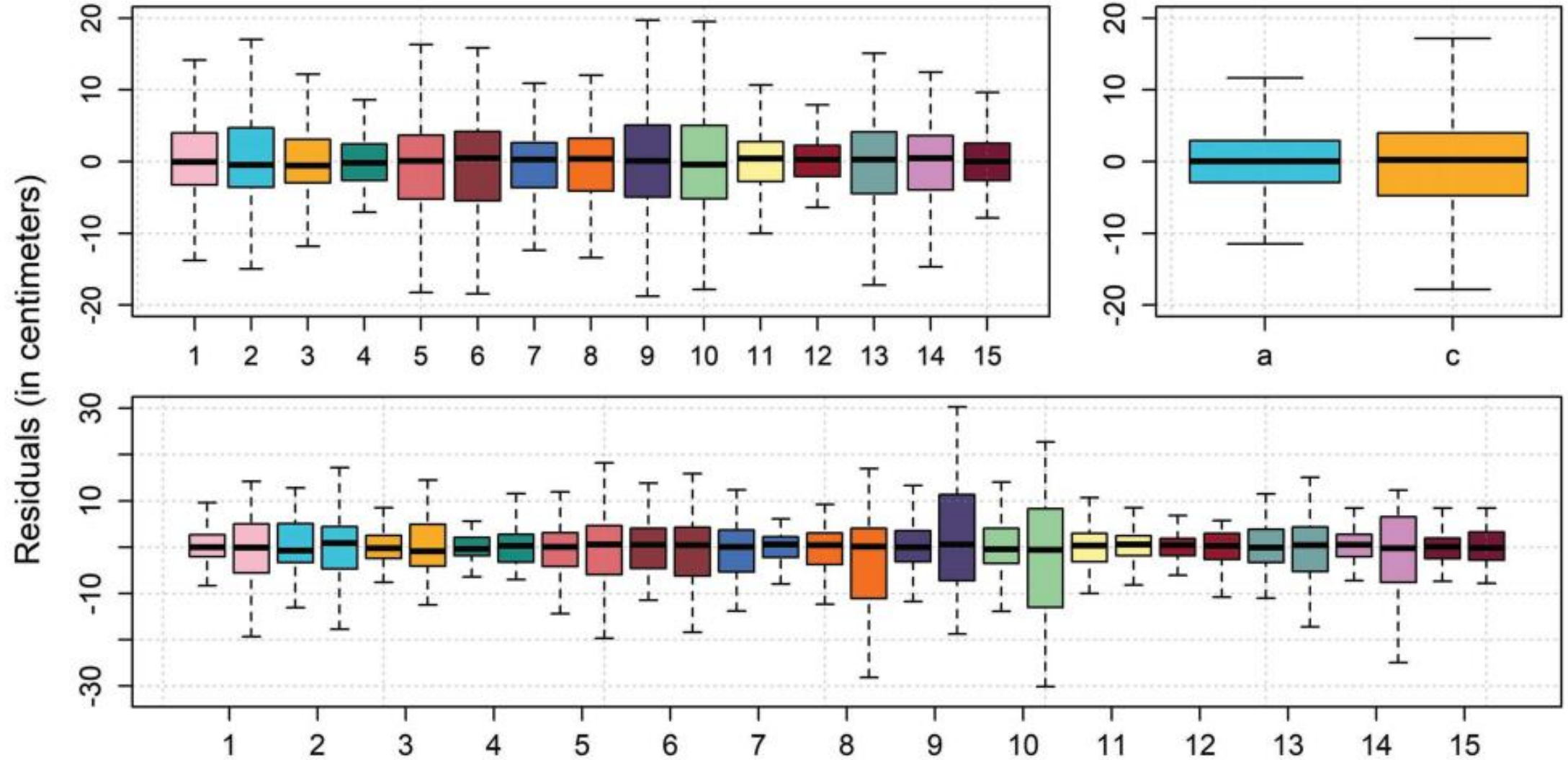
$$\text{height}_{[i]} \sim \mathcal{N}(\mu_{[i]}, \sigma_{[i]})$$

$$\mu_{[i]} = x_1 + x_2 + \dots + x_p$$

$$\sigma_{[i]} = x_{\sigma 1} + x_{\sigma 2} + \dots + x_{\sigma p}$$



Heteroskedasticity



Description of Our Model

Capítulo 7

`height ~ A + G + A:G + (A + G + A:G|L) + (1|S)`

Juntos en un
solo modelo

Nuevo componente,
similar al capítulo 5.

`sigma ~ A`

`model_formula = brms::bf(height ~ A*G + (A*G|L) + (1|S),
sigma ~ A)`

Description of Our Model

$$\begin{aligned} \text{height}_{[i]} &\sim t(v, \mu_{[i]}, \sigma_{[i]}) \\ \mu_{[i]} &= \text{Intercept} + A + G + A:G + \\ &L_{[L[i]]} + A:L_{[L[i]]} + G:L_{[L[i]]} + A:G:L_{[L[i]]} + S_{[S[i]]} \end{aligned}$$

$$\log(\sigma_{[i]}) = \text{Intercept}_{\sigma} + A_{\sigma}$$

Priors:

$$S_{[\cdot]} \sim t(3, 0, \sigma_S)$$

$$\begin{bmatrix} L_{[\cdot]} \\ A:L_{[\cdot]} \\ G:L_{[\cdot]} \\ A:G:L_{[\cdot]} \end{bmatrix} \sim \text{MVNormal} \left(\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \Sigma \right)$$

$$\text{Intercept} \sim t(3, 156, 12)$$

$$A, G, A:G \sim t(3, 0, 12)$$

$$\sigma_L, \sigma_{A:L}, \sigma_{G:L}, \sigma_{A:G:L}, \sigma_S \sim t(3, 0, 12)$$

$$v \sim \text{gamma}(2, 0.1)$$

$$R \sim \text{LKJCorr}(2)$$

$$\text{Intercept}_{\sigma} \sim N(0, 1.5)$$

$$A_{\sigma} \sim N(0, 1.5)$$

$$\text{height}_{[i]} \sim t(v, \mu_{[i]}, \sigma_{[i]})$$

$$\sigma_{[i]} = \text{Intercept}_{\sigma} + A_{\sigma}$$

$$\text{Intercept}_{\sigma} \sim t(3, 0, 1.5)$$

$$A_{\sigma} \sim t(3, 0, 1.5)$$

Fitting the Model

```
model_formula = brms::bf(height ~ A*G + (A*G|L) + (1|S),  
                          sigma ~ A)   
  
priors =  
  c(set_prior("student_t(3, 156, 12)", class = "Intercept"),  
    set_prior("student_t(3, 0, 12)", class = "b"),  
    set_prior("student_t(3, 0, 12)", class = "sd"),  
    set_prior("gamma(2, 0.1)", class = "nu"),  
    set_prior("normal(0, 1.5)", class = "Intercept", dpar = "sigma"),   
    set_prior("normal(0, 1.5)", class = "b", dpar = "sigma"),  
    set_prior("lkj_corr_cholesky(2)", class = "cor"))   
  
prior_A_sigma =  
  brms::brm(model_formula, data = exp_data, chains = 4, cores = 4,  
            warmup = 1000, iter = 3500, thin = 2, family="student",  
            prior = priors, sample_prior = "only")
```


Inspecting the Fixed Effects

```
fixef (model_A_sigma)
##               Estimate Est.Error      Q2.5      Q97.5
## Intercept      158.383    1.10922  156.193  160.6196
## sigma_Intercept    1.724    0.03265    1.659    1.7881
## A1              11.271    1.19021    8.915   13.6901
## G1              -3.067    0.59091   -4.271   -1.9084
## A1:G1            -1.528    0.43464   -2.403   -0.6654
## sigma_A1         -0.247    0.02199   -0.290   -0.2036
```



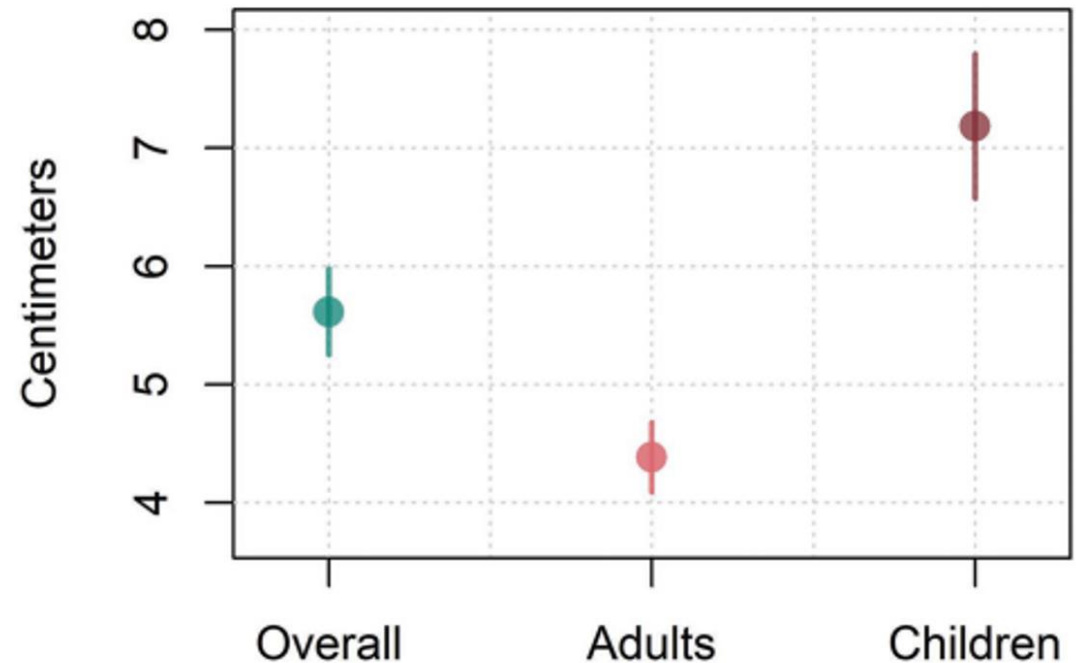
En unidades logarítmicas. Debe exponenciar valores para obtener las unidades originales, es decir, $\sigma = \exp(\text{sigma_Intercept})$

Recovering predicted Sigmas

Debe combinar parámetros antes de exponenciar. De lo contrario, los resultados serán totalmente erróneos.

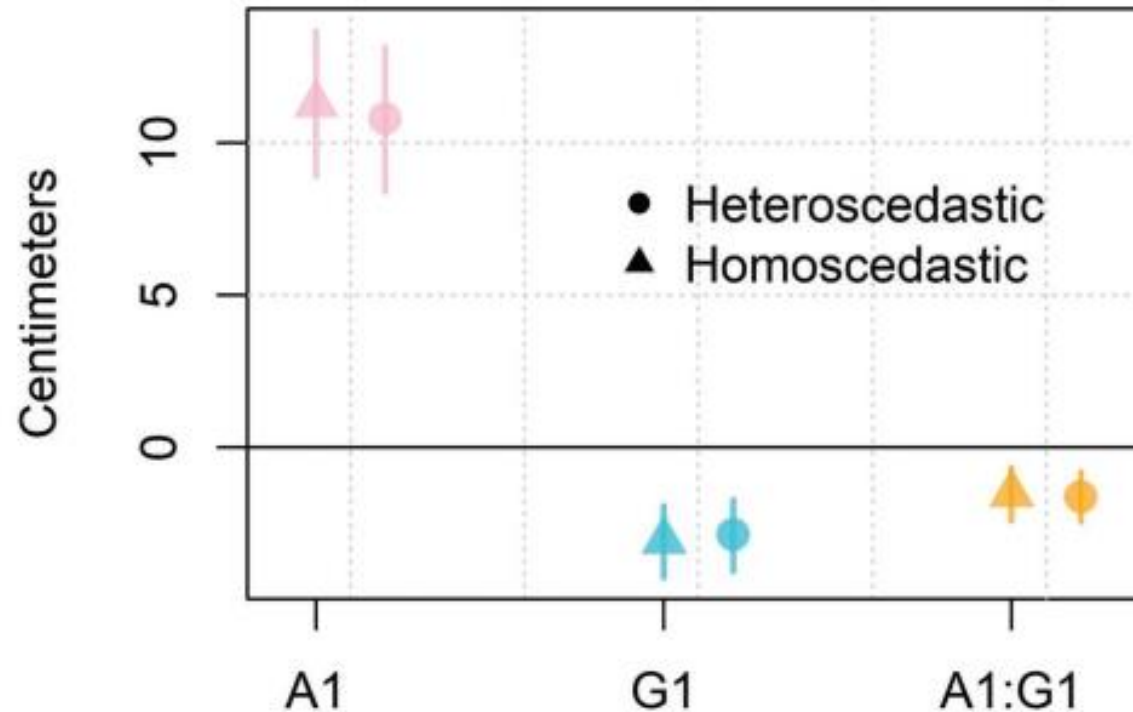
```
sigmas = short_hypothesis(  
  model_A_sigma,  
  c("exp(sigma_Intercept) = 0",           # overall sigma  
    "exp(sigma_Intercept + sigma_A1) = 0", # adult sigma  
    "exp(sigma_Intercept - sigma_A1) = 0")) # child sigma
```

```
sigmas[, -5]  
##      Estimate Est.Error  Q2.5 Q97.5  
## H1      5.609    0.1830  5.253  5.978  
## H2      4.382    0.1526  4.085  4.680  
## H3      7.184    0.3115  6.573  7.793
```



Model Comparison

```
model_interaction = add_criterion(model_interaction, "loo")
model_A_sigma = add_criterion(model_A_sigma, "loo")
loo_compare (model_interaction, model_A_sigma)
##                               elpd_diff se_diff
## model_A_sigma                 0.0         0.0
## model_interaction            -53.7        12.8
```



A 'Complex' Model

- Nuestro último modelo solo tenía un "efecto fijo" para la edad aparente.
- Si estuviéramos prediciendo la media, querríamos incluir efectos aleatorios para el oyente.
- También podemos incluirlos en nuestra predicción de sigma.

```
sigma ~ A
```

```
sigma ~ A + (A|L)
```

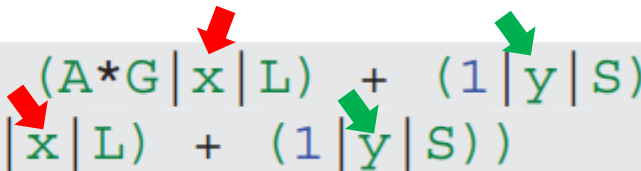
Our Model Formula

- Cuando tenemos "los mismos" efectos aleatorios en todas las fórmulas, debemos hacérselo saber a nuestro modelo.

```
model_formula = brms::bf(height ~ A*G + (A*G|L) + (1|S),  
                           sigma ~ A + (A|L))
```

- Hacemos esto colocando el mismo indicador entre lineas (|aquí|) antes de un efecto aleatorio.

```
model_formula = brms::bf(height ~ A*G + (A*G|x|L) + (1|y|S),  
                           sigma ~ A + (A|x|L) + (1|y|S))
```



- ¡Nuestro modelo estimará la correlación de los efectos aleatorios entre las variables predichas!

Model Description

$$\begin{aligned} \text{height}_{[i]} &\sim \text{N}(\mu_{[i]}, \sigma_{[i]}) \\ \mu_{[i]} &= \text{Intercept} + A + G + A : G + \\ &L_{[L[i]]} + A : L_{[L[i]]} + G : L_{[L[i]]} + A : G : L_{[L[i]]} + S_{[S[i]]} \\ \log(\sigma_{[i]}) &= \text{Intercept}_{\sigma} + A_{\sigma} + A_{\sigma} : L_{\sigma[L[i]]} + L_{\sigma[L[i]]} \end{aligned}$$

Priors :

$$\begin{bmatrix} L_{[\cdot]} \\ A : L_{[\cdot]} \\ G : L_{[\cdot]} \\ A : G : L_{[\cdot]} \\ L_{\sigma[\cdot]} \\ A_{\sigma} : L_{\sigma[\cdot]} \end{bmatrix} \sim \text{MVNormal} \left(\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \Sigma \right)$$

$$S_{[\cdot]} \sim \text{N}(0, \sigma_S)$$

$$\text{Intercept} \sim \text{t}(3, 156, 12)$$

$$A \sim \text{t}(3, 0, 12)$$

$$\sigma, \sigma_L, \sigma_{A:L}, \sigma_S \sim \text{t}(3, 0, 12)$$

$$\text{Intercept}_{\sigma} \sim \text{N}(0, 1.5)$$

$$A_{\sigma} \sim \text{N}(0, 1.5)$$

$$\sigma_{L_{\sigma}}, \sigma_{A_{\sigma}:L_{\sigma}} \sim \text{N}(0, 1.5)$$

$$R \sim \text{LKJCorr}(2)$$

Fitting the Model

```
# Fit the model yourself
model_formula = brms::bf(height ~ A*G + (A*G|x|L) + (1|S),
                          sigma ~ A + (A|x|L))

priors =
  c(set_prior("student_t(3, 156, 12)", class = "Intercept"),
    set_prior("student_t(3, 0, 12)", class = "b"),
    set_prior("student_t(3, 0, 12)", class = "sd"),
    set_prior("gamma(2, 0.1)", class = "nu"),
    set_prior("normal(0, 1.5)", class = "Intercept", dpar = "sigma"),
    set_prior("normal(0, 1.5)", class = "b", dpar = "sigma"),
    set_prior("normal(0, 1.5)", class = "sd", dpar = "sigma"),
    set_prior("lkj_corr_cholesky (2)", class = "cor"))

model_A_L_sigma =
  brms::brm (model_formula, data = exp_data, chains = 4, cores = 4,
            warmup = 1000, iter = 3500, thin = 2, family="student",
            prior = priors)
```


Inspecting the Results

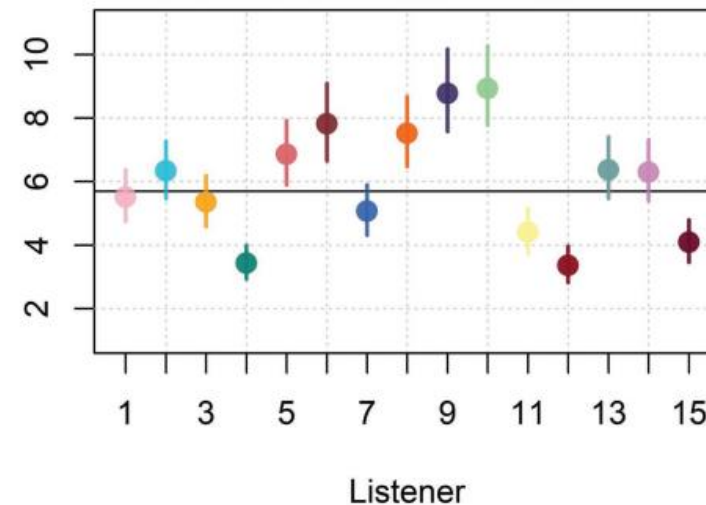
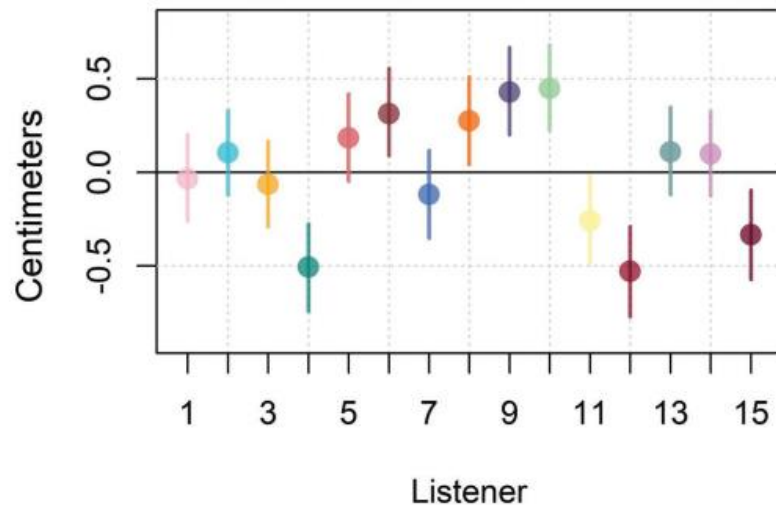
```
bmbb::short_summary (model_A_L_sigma)
## ...
## sd(sigma_Intercept)          0.36      0.08      0.24      0.54
## sd(sigma_A1)                 0.17      0.04      0.10      0.27
## ...
## cor(A1,sigma_A1)             -0.29      0.22     -0.67      0.16
## cor(G1,sigma_A1)             0.22      0.24     -0.27      0.65
## cor(A1:G1,sigma_A1)          0.12      0.24     -0.35      0.58
## cor(sigma_Intercept,sigma_A1) -0.44      0.21     -0.79      0.05
## ...
##
## Population-Level Effects:
##               Estimate Est.Error l-95% CI u-95% CI
## Intercept         158.29      1.10   156.15   160.51
## sigma_Intercept     1.74      0.10     1.55     1.93
## A1                 11.28      1.18     8.98    13.64
## G1                 -2.92      0.57    -4.06    -1.80
## A1:G1              -1.63      0.42    -2.47    -0.77
## sigma_A1           -0.23      0.05    -0.33    -0.14
##
## Family Specific Parameters:
##      Estimate Est.Error l-95% CI u-95% CI
## nu      8.07      1.54     5.71    11.72
```

Listener-dependent Error Terms

```
# listener random effects for sigma
log_sigmas_centered = short_hypothesis(model_A_L_sigma, "sigma_Intercept=0",
                                       scope = "ranef", group="L")

# the sum of the sigma intercept and the listener sigma random effect
log_sigmas = short_hypothesis(model_A_L_sigma, "sigma_Intercept=0",
                              scope = "coef", group="L")

# the exponent of the sum of the sigma intercept
# and the listener sigma random effect
sigmas = short_hypothesis(model_A_L_sigma, "exp(sigma_Intercept)=0",
                          scope = "coef", group="L")
```



Model Comparison

- Nuestro nuevo modelo es mucho mejor.

```
model_A_L_sigma = add_criterion(model_A_L_sigma, "loo")
```

```
loo_compare (model_interaction, model_A_sigma, model_A_L_sigma)
##               elpd_diff se_diff
## model_A_L_sigma      0.0      0.0
## model_A_sigma     -122.1     15.6
## model_interaction -175.8     20.8
```

Answering our Research Questions

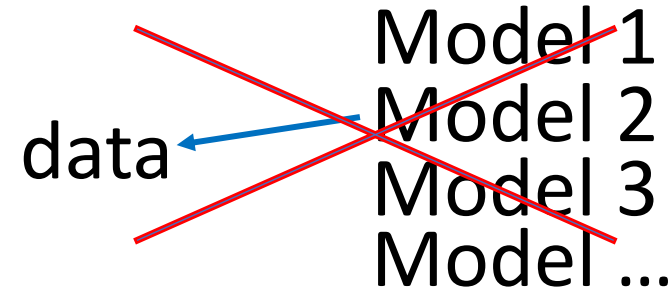
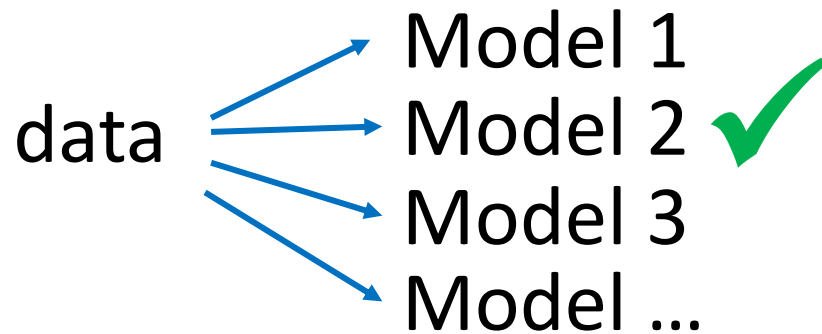
(Q1) Does our error standard deviation vary as a function of apparent speaker age?

(Q2) Does our error standard deviation vary as a function of the listener?

- Sí, y sí.
- Algunas preguntas más difíciles son: ¿Cuál es el modelo "mejor/real"? ¿Qué modelo debemos reportar?

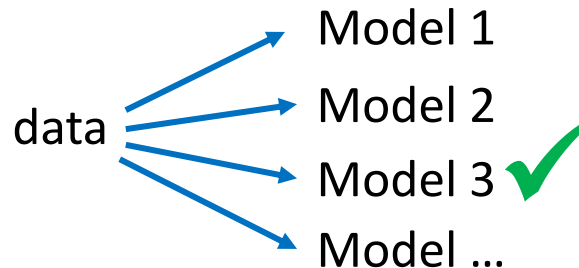
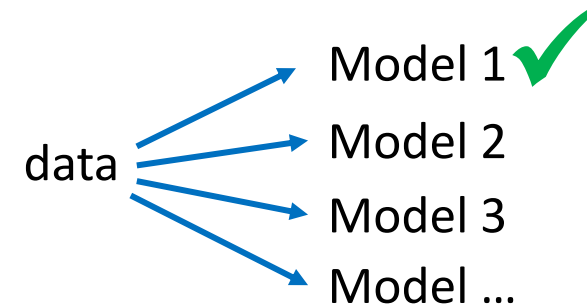
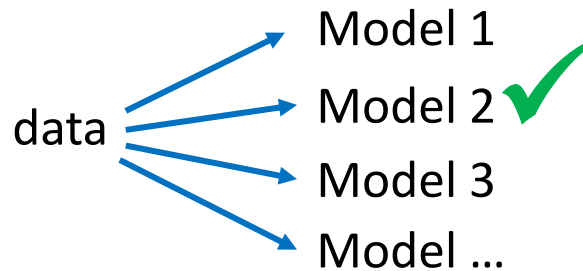
Underdetermination

- Dadas algunas observaciones, siempre hay múltiples interpretaciones/modelos diferentes.



Researcher Degrees of Freedom

- Las decisiones que toma un investigador durante el diseño o el análisis que afectan al modelo utilizado y a las conclusiones a las que se llega.

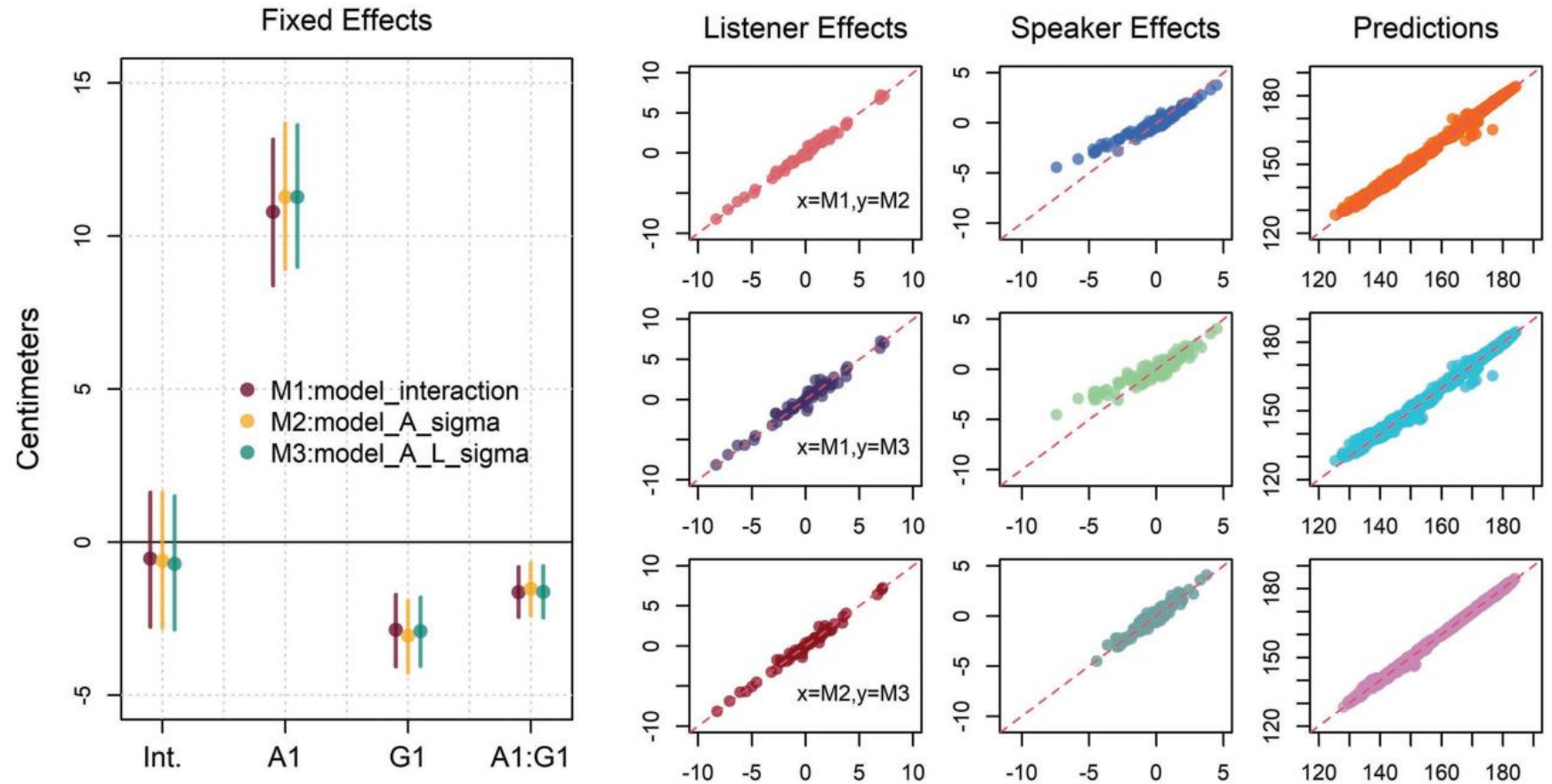


Which Model To Report?

- Nuestros modelos representan información diferente.
- Cuando consideras cuál presentar, ten en cuenta dos cosas:
 1. ¿Qué parámetros/información nos interesan?
 2. ¿Cómo difieren entre los modelos?
 - 2a. Si difieren, ¿por qué?

Which Model To Report?

- Los efectos fijos son más o menos los mismos.
- Los efectos de los hablantes difieren en los modelos heterocedásticos.
- ¿Te importa la variación de los errores?



Should You fit the Model?

- A medida que avanzamos, *se pueden* ajustar modelos cada vez más complicados.
 - Pero, ¿*deberías* hacerlo?
- Dos cosas de las que preocuparse:
 - Identificabilidad: la capacidad de estimar valores únicos e independientes para todos sus parámetros.
 - Soporte: Tener suficientes datos para estimar de manera realista todos sus parámetros.

Collinearity

- Un conjunto de vectores predictores es linealmente independiente cuando no hay ningún vector de números distintos de cero que se pueda usar para combinar nuestros predictores de modo que siempre sean iguales a cero.

$$0 = x_1 \cdot a_1 + x_2 \cdot a_2 + \dots + x_n \cdot a_n$$



Si esto es posible, los predictores x no son linealmente dependientes.

Collinearity

- No se puede ajustar un modelo utilizando la altura en metros y la altura en centímetros, y estimar ambos efectos de forma independiente.

```
exp_data$vtl_m = exp_data$vtl / 100
```

```
exp_data$vtl + exp_data$vtl_m * -100
```

- ¡Estos predictores son linealmente dependientes!

$$x_1 \cdot a_1 + x_2 \cdot a_2 = 0$$

$$\text{vtl} \cdot 1 + \text{vtl}_m \cdot -100 = 0$$

Collinearity

```
model_bad_1 =  
  brms::brm (height ~ vtl_m + vtl, data = exp_data, chains = 4, cores = 4,  
    warmup = 1000, iter = 3500, thin = 2,  
    prior = c(brms::set_prior("normal(176, 50)", class = "Intercept"),  
      brms::set_prior("normal(0, 15)", class = "sigma")))
```

Population-Level Effects:

##	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
## Intercept	45.69	2.37	41.17	50.47	1.00	4031	4425
## vtl_m	1383.91	246800.93	-531293.80	343951.54	2.12	5	13
## vtl	-5.28	2468.01	-3430.88	5321.56	2.12	5	13

Semi-Collinearity

- La dependencia lineal es binaria. La correlación es una medida continua de la dependencia lineal (básicamente).

```
cor (exp_data$vtl_m, exp_data$vtl)
## [1] 1
```

- ¿Qué pasa si nuestros predictores dependen casi linealmente?

```
set.seed(1)
exp_data$vtl_m_noise = exp_data$vtl_m +
  rnorm (length(exp_data$vtl_m), 0, sd(exp_data$vtl_m) / 10)

cor (exp_data$vtl, exp_data$vtl_m_noise)
## [1] 0.9946
```

Semi-Collinearity: Not that Bad

```
model_bad_2 =  
  brms::brm (height ~ vtl_m_noise + vtl, data = exp_data, chains = 4,  
            cores = 4, warmup = 1000, iter = 3500, thin = 2, prior = priors)
```

Population-Level Effects:

##	Estimate	Est.Error	1-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
## Intercept	45.75	2.36	41.16	50.35	1.00	4523	4087
## vtl_m_noise	-201.86	175.11	-544.11	139.03	1.00	3504	3697
## vtl	10.57	1.76	7.14	14.01	1.00	3439	3866

Semi-Collinearity: Could be Better

```
model_good =  
  brms::brm (height ~ vtl, data = exp_data, chains = 4, cores = 4,  
    warmup = 1000, iter = 3500, thin = 2,  
    prior = c(brms::set_prior("normal(176, 50)", class = "Intercept"),  
      brms::set_prior("normal(0, 15)", class = "sigma")))
```

Population-Level Effects:

##		Estimate	Est.Error	1-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
##	Intercept	45.72	2.38	41.04	50.40	1.00	5052	4679
##	vtl	8.55	0.18	8.21	8.90	1.00	5072	4831

```
model_bad_2 =
```

```
  brms::brm (height ~ vtl_m_noise + vtl, data = exp_data, chains = 4,  
    cores = 4, warmup = 1000, iter = 3500, thin = 2, prior = priors)
```

Population-Level Effects:

##		Estimate	Est.Error	1-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
##	Intercept	45.75	2.36	41.16	50.35	1.00	4523	4087
##	vtl_m_noise	-201.86	175.11	-544.11	139.03	1.00	3504	3697
##	vtl	10.57	1.76	7.14	14.01	1.00	3439	3866

Predictable Values for Predictors

- No se pueden ajustar modelos en los que el valor de un predictor categórico se pueda adivinar en función de las vlues de otros predictores.
- De nuevo, se trata de un problema de dependencia lineal.
- Por esto no podemos estimar ambos niveles de un factor de dos grupos.

```
x = cbind (intercept=rep(1,4), x1=rep(c(1,0),2),  
x2=rep(c(0,1),2))  
x  
##           intercept x1 x2  
## [1,]             1  1  0  
## [2,]             1  0  1  
## [3,]             1  1  0  
## [4,]             1  0  1
```

```
x[,1] + x[,2]*(-1) + x[,3]*(-1)  
## [1] 0 0 0 0
```


Predictable Values for Predictors

- Y también por qué no podemos estimar todos los niveles de un factor de cuatro niveles.

```
x = cbind (intercept=rep(1,4), C1=c(1,0,0,0), C2=c(0,1,0,0),  
           C3=c(0,0,1,0), C4=c(0,0,0,1))
```

X

##		<i>intercept</i>	<i>C1</i>	<i>C2</i>	<i>C3</i>	<i>C4</i>
##	[1,]	1	1	0	0	0
##	[2,]	1	0	1	0	0
##	[3,]	1	0	0	1	0
##	[4,]	1	0	0	0	1

```
x[,1] + x[,2]*(-1) + x[,3]*(-1) + x[,4]*(-1) + x[,5]*(-1)  
## [1] 0 0 0 0
```

Predictable Values for Predictors

- Y también por qué no podemos incluir el grupo, la edad y el género.

```
x = cbind (intercept=rep(1,4), C1=c(1,0,0,0), C2=c(0,1,0,0),  
          C3=c(0,0,1,0), A1=c(0,0,1,1), G1=c(0,1,0,1), A1G1=c(1,0,0,1))
```

```
X  
##      intercept C1 C2 C3 A1 G1 A1G1  
## [1,]         1  1  0  0  0  0     1  
## [2,]         1  0  1  0  0  1     0  
## [3,]         1  0  0  1  1  0     0  
## [4,]         1  0  0  0  1  1     1
```

```
x[,1]*1 + x[,2]*(-1) + x[,3]*(-1) + x[,5]*(-1)  
## [1] 0 0 0 0
```

```
x[,1]*1 + x[,3]*(-1) + x[,4]*(-1) + x[,7]*(-1)  
## [1] 0 0 0 0
```

Predictable Values for Predictors

```
model_bad_3 =  
  brms::brm (height ~ C + A*G, data = exp_data, chains = 4, cores = 4,  
    warmup = 1000, iter = 3500, thin = 2,  
    prior = c(brms::set_prior("normal(176, 15)", class = "Intercept"),  
      brms::set_prior("normal(0, 15)", class = "sigma")))
```

Population-Level Effects:

##	Estimate	Est.Error	1-95% CI	u-95% CI	Rhat	Bulk ESS	Tail ESS
## Intercept	157.98	0.20	157.57	158.35	1.06	61	78
## C1	1066.31	2604.72	-4431.79	5158.68	2.07	5	18
## C2	496.91	1751.32	-2543.19	3708.83	1.87	6	13
## C3	74.39	2086.76	-4113.31	2801.74	1.99	5	21
## A1	793.68	1363.24	-2383.78	3019.96	1.90	6	12
## G1	567.63	2171.69	-4107.90	3364.23	2.04	5	18
## A1:G1	283.89	1112.60	-2295.30	2141.73	2.26	5	18

Esto no es ideal.

Saturated Models

- Los modelos saturados tienen un parámetro para cada observación.
- Sin 'shrinkage', esto significa que no hay variación aleatoria en el modelo, es decir, no se puede estimar el error.

```
exp_data$S = factor (exp_data$S)
exp_data$L = factor (exp_data$L)

model_bad_4 =
  brms::brm (height ~ S*L, data = exp_data, chains = 4, cores = 4,
    warmup = 1000, iter = 3500, thin = 2,
    prior = c(brms::set_prior("normal(176, 15)", class = "Intercept"),
      brms::set_prior("normal(0, 15)", class = "sigma")))
```



Este modelo bloquea mi sesión de R.

Nearly-Saturated Models

- Un modelo puede estar "casi" saturado.
- ¿Qué pasa si tiene 2 observaciones por hablante por oyente?
Puedes ajustar este modelo (probablemente), pero ¿deberías?
- ¿Cuál es tu n ?
- Piense en su n para cada parámetro individual en lugar de en general.

Exercises: Week 7

Use the data in 'exp_ex' to do one of the following. You may also use your own data to answer a related question. In any case, describe the model, present and explain the results. More requirements:

- You must include a model at least this complex: $y \sim A*B + (A*B|L)$, meaning two factors and an interaction.
- Fit a new model, not like in the book and not like for week 6.
- Include at least two non-book figures.
- Recreate the predicted group means and report them.
- Report and interpret at least one simple effect.